CHAPTER 1

이미지 분류

성명 : 최승호
이메일 : jcn99250@naver.com

학습 목표

- 딥러닝을 이용한 이미지 분류에 대해서 이해합니다.

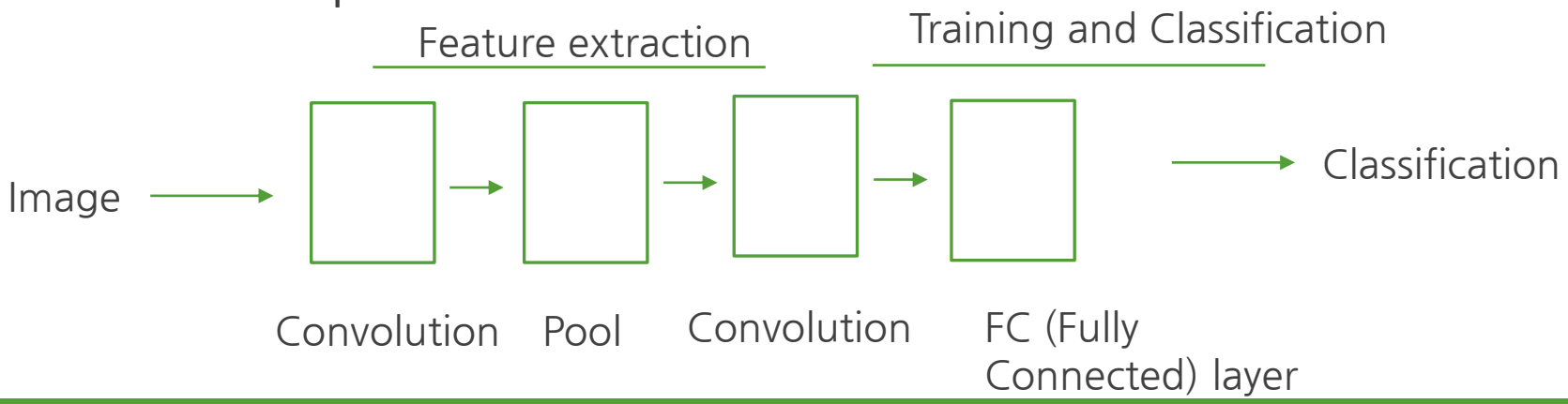
9.1 딥 러닝을 이용한 이미지 분류

- 딥러닝 모델 예측 과정:
  - 해당 영상을 딥러닝 모델에 입력하면 해당 영상에 대한 정답이 모델에서 예측이 되어서 나오는 과정
  - 이미지 분류 :
    - 사전에 정의한 카테고리 내에서 딥러닝 모델 예측이 수행 되고 오차를 이용해서 딥러닝 모델이 학습하는 것
    - 학습된 모델을 가지고 이미지에 대해서 테스트하는 작업



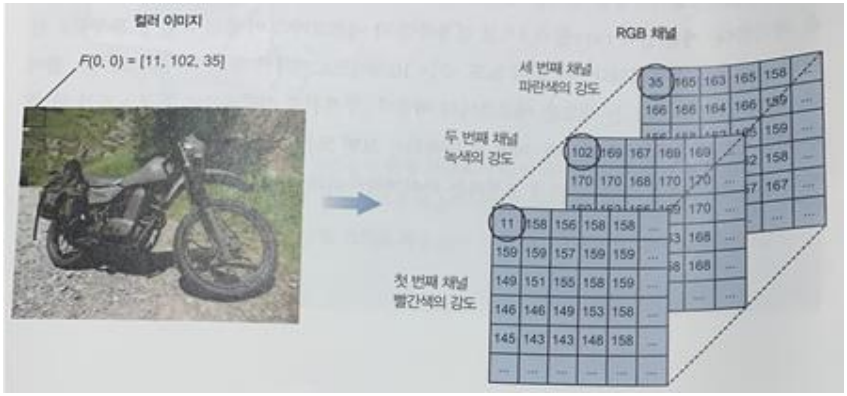
9.1 딥 러닝을 이용한 이미지 분류

- 이미지 분류에 대표적인 모델:
  - CNN 모델 (Convolutional Neural Network) :
    - Convolution과 Pool 연산을 반복적으로 사용하면서 불변하는 특징을 찾고 그 특징을 입력데이터로 Fully Connected layer에 보내 Classification 을 수행함
  - Concept :



9.1 딥 러닝을 이용한 이미지 분류

- 컬러 이미지의 합성곱 연산:
  - 앞서 살펴본 cnn은 gray scale 이미지를 픽셀값의 2차원 행렬로 다룬다고 설명함
  - 실제 이미지는 픽셀값이 담긴 2차원 행렬, 각 픽셀값은 원색의 강도로 표현됨, 즉, 너비, 높이, 깊이를 가진 3차원 형태의 행렬 형태, RGB이미지라고 하면 깊이는 채널별로 하나씩 3



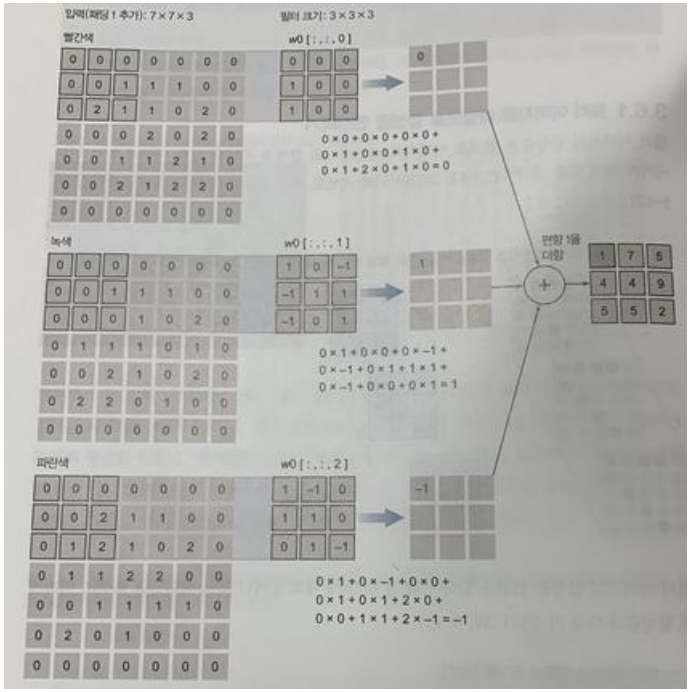
계산함

9.1 딥 러닝을 이용한 이미지 분류

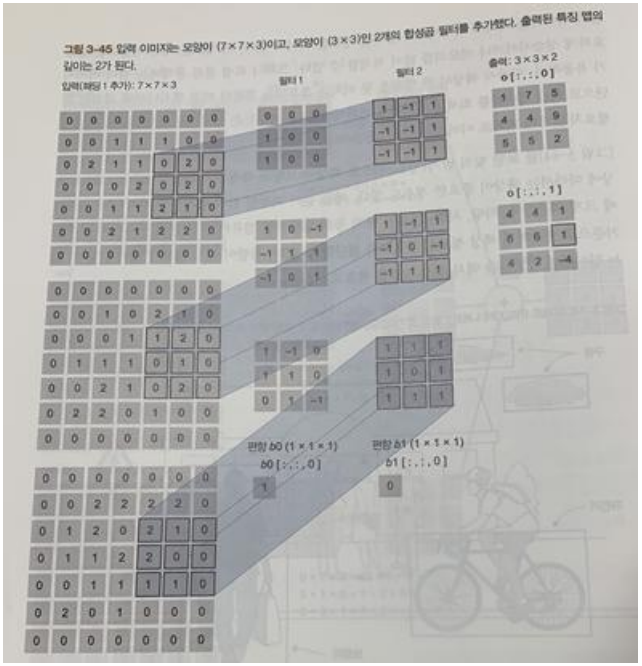
- 컬러 이미지의 합성곱 연산:
  - 컬러 이미지의 합성 곱은 입력 이미지 위로 이동시키며 특징 맵을 계산하면 됨
  - 컬러 이미지에 합성 곱 커널을 이동시키며 특징 맵을 계산하려면 커널도 3차원이어야 함
  - 컬러 이미지의 합성 곱 연산
    - 색상 채널별로 별도의 필터를 가짐
    - 각 필터는 해당 하는 채널 이미지의 위를 이동함, 겹쳐진 픽셀끼리 픽셀 값을 곱하고 곱한 값을 모두 합해 전과 마찬가지로 픽셀 값을 계산함
    - 세 채널의 픽셀 값을 모아 특징 맵의 픽셀 값을 계산함

9.1 딥 러닝을 이용한 이미지 분류

- 컬러 이미지의 합성곱 연산:
  - 컬러 이미지의 합성 곱 연산(계속):



필터를 이용한 연산 과정



2개의 필터를 이용했을때 Feature map 생성되는 과정

9.1 딥 러닝을 이용한 이미지 분류

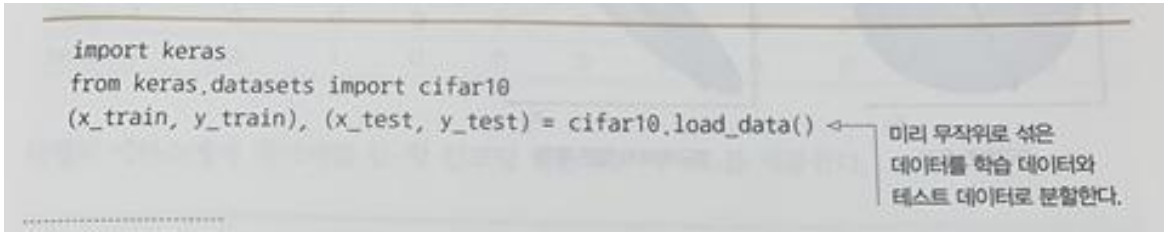
- 컬러 이미지 분류 문제
  - CNN을 학습해서 CIFAR-10 데이터 셋 이미지를 분류하고자 함
  - CIFAR-10 데이터 셋
    - 컴퓨터 비전 분야에서 널리 알려진 사물인식 문제 데이터 셋
    - 32by32 크기의 컬러 이미지 6만 장으로 구성
    - 클래스당 6천 장씩 10가지 클래스로 분류

9.1 딥 러닝을 이용한 이미지 분류

- 컬러 이미지 분류 문제
  - 컬러 이미지 분류하기 위한 단계 요약
    - 1단계 : 데이터 셋 읽어 들이기
    - 2단계 : 이미지 전 처리
    - 3단계 : 모델 구조 정의하기
    - 4단계 : 모델 컴파일 하기
    - 5단계 : 모델 학습하기
    - 6단계: val acc 가 높은 모델 사용
    - 7단계 : 모델 평가하기

9.1 딥 러닝을 이용한 이미지 분류

- 컬러 이미지 분류 문제
  - 컬러 이미지 분류하기 위한 단계
    - 1단계 : 데이터 셋 읽어 들이기
      - 데이터 셋을 읽어 들여 학습 /검증 / 테스트로 분할 해야함
      - 케라스 라이브러리에 CIFAR 데이터 셋을 읽어 들일 수 있는 load_data()메서드가 제공함



9.1 딥 러닝을 이용한 이미지 분류

- 컬러 이미지 분류 문제
  - 컬러 이미지 분류하기 위한 단계
    - 2단계 : 이미지 전 처리
      - 문제와 데이터에 따라 다르겠지만 일반적으로는 모델의 학습을 시작하기 전에 약간의 데이터 클린징과 전처리르 거침
      - **이미지의 픽셀값 정규화 하기**

```
x_train = x_train.astype('float32')/255
x_test = x_test.astype('float32')/255
```

픽셀값의 구간을 [0, 255]에서 [0, 1]로 바꾸어 정규화한다.

9.1 딥 러닝을 이용한 이미지 분류

- 컬러 이미지 분류 문제
  - 컬러 이미지 분류하기 위한 단계
    - 2단계 : 이미지 전 처리
      - **레이블 준비하기 (원-핫 인코딩)**
        - 데이터셋에 포함된 이미지는 그 이미지 유형에 따라 레이블이 부여됨
        - CIFAR-10을 예로 들면 [airplane, automobile, bird, cat, deer, dog, frog, horse, ship, truck]등 10개의 레이블이 부여됨
        - 텍스트 레이블 역시 컴퓨터가 다룰 수 있는 숫자 형태로 변환해야 함
        - 이때 원-핫 인코딩(one-hot encoding)이라는 방법을 사용

9.1 딥 러닝을 이용한 이미지 분류

- 컬러 이미지 분류하기 위한 단계
  - 2단계 : 이미지 전 처리
    - **레이블 준비하기 (원-핫 인코딩)**
      - 원-핫 인코딩(one-hot encoding)이라는 방법 예시

이미지	레이블
Image_1	dog
Image_2	automobile
Image_3	airplane
Image_4	truck
Image_5	bird

이 데이터에 원-핫 인코딩을 적용하면 다음과 같은 데이터가 된다.

	airplane	bird	cat	deer	dog	frog	horse	ship	truck	automobile
Image_1	0	0	0	0	1	0	0	0	0	0
Image_2	0	0	0	0	0	0	0	0	0	1
Image_3	1	0	0	0	0	0	0	0	0	0
Image_4	0	0	0	0	0	0	0	0	1	0
Image_5	0	1	0	0	0	0	0	0	0	0

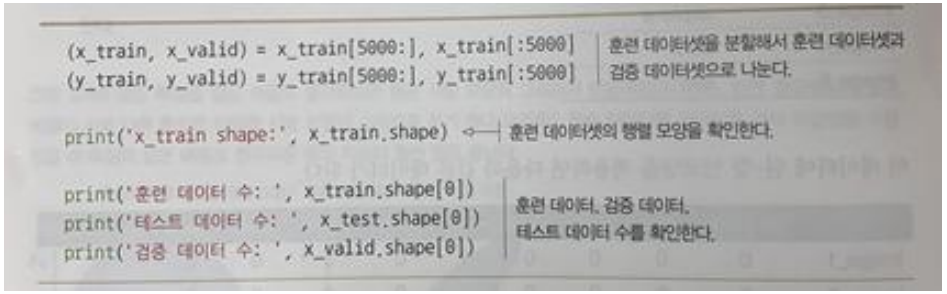
      - 케라스 소스 코드

```
from keras.utils import np_utils

num_classes = len(np.unique(y_train))   <- 레이블에 원-핫 인코딩 적용
y_train = keras.utils.to_categorical(y_train, num_classes)
y_test = keras.utils.to_categorical(y_test, num_classes)
```

9.1 딥 러닝을 이용한 이미지 분류

- 컬러 이미지 분류하기 위한 단계
  - 2단계 : 이미지 전 처리
    - **훈련 데이터와 테스트 데이터 분할**
      - 훈련데이터 : 모델을 학습하는데 사용하는 데이터
      - 검증 데이터 : 훈련 데이터에 치우치지 않도록 하기 위한 별도의 데이터
      - 테스트 데이터 : 모델의 성능을 최종 판단하기 위해 사용되는 데이터
      - 케라스 구현



```
(x_train, x_valid) = x_train[5000:], x_train[:5000]
(y_train, y_valid) = y_train[5000:], y_train[:5000]

print('x_train shape:', x_train.shape)

print('훈련 데이터 수: ', x_train.shape[0])
print('테스트 데이터 수: ', x_test.shape[0])
print('검증 데이터 수: ', x_valid.shape[0])
```

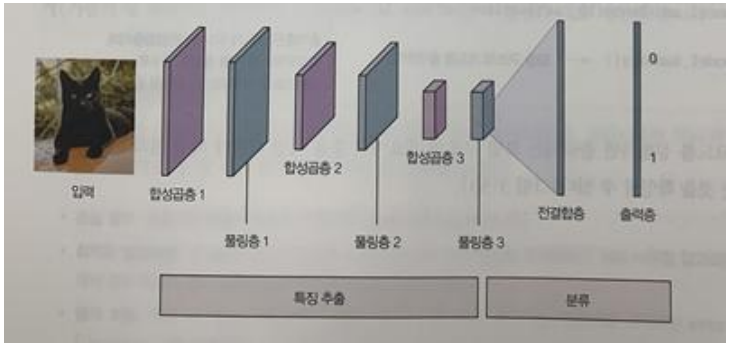
훈련 데이터셋을 분할해서 훈련 데이터셋과 검증 데이터셋으로 나눈다.

훈련 데이터셋의 행렬 모양을 확인한다.

훈련 데이터, 검증 데이터, 테스트 데이터 수를 확인한다.

9.1 딥 러닝을 이용한 이미지 분류

- 컬러 이미지 분류 문제
  - 컬러 이미지 분류하기 위한 단계
    - 3단계 : 모델 구조 정의하기
      - 신경망 구조를 설계하는 것은 기존 유명한 신경망 구조를 따라서 만들어 보기 후 하나씩 수정해보면서 영향성에 대한 분석을 진행하기
      - AlexNet의 간략 버전을 구현해보고자 함



9.1 딥 러닝을 이용한 이미지 분류

- 3단계 : 모델 구조 정의하기 소스코드 및 실행 결과

```
from keras.models import Sequential
from keras.layers import Conv2D, MaxPooling2D, Flatten, Dense, Dropout

model = Sequential()
model.add(Conv2D(filters=16, kernel_size=2, padding='same',
    activation='relu', input_shape=(32, 32, 3)))
model.add(MaxPooling2D(pool_size=2))

model.add(Conv2D(filters=32, kernel_size=2, padding='same',
    activation='relu'))
model.add(MaxPooling2D(pool_size=2))

model.add(Conv2D(filters=64, kernel_size=2, padding='same',
    activation='relu'))
model.add(MaxPooling2D(pool_size=2))

model.add(Dropout(0.3))

model.add(Flatten())

model.add(Dense(500, activation='relu'))
model.add(Dropout(0.4))

model.add(Dense(10, activation='softmax'))

model.summary()
```

첫 번째 합성곱층과 풀링층. 입력 모양(input_shape)은 첫 번째 합성곱층에만 지정한다.

두 번째 합성곱층과 풀링층. ReLU를 활성화 함수로 사용한다.

세 번째 합성곱층과 풀링층

과적합을 피하기 위한 드롭아웃층. 드롭아웃 비율을 30%로 설정했다.

특징 맵을 1차원 특징 벡터로 변환한다.

첫 번째 전결합층

드롭아웃 비율이 40%인 두 번째 드롭아웃층

출력층은 노드가 10개인 전결합층이며, 소프트맥스 함수를 활성화 함수로 사용하므로 각 클래스의 확률을 출력한다.

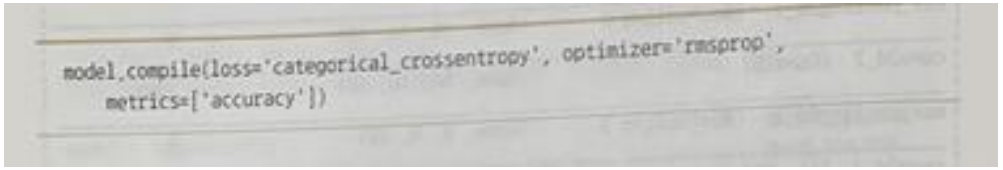
모델 구조의 개요를 출력한다.

Layer (type)	Output Shape	Param #
conv2d_1 (Conv2D)	(None, 32, 32, 16)	208
max_pooling2d_1 (MaxPooling 2	(None, 16, 16, 16)	0
conv2d_2 (Conv2D)	(None, 16, 16, 32)	2080
max_pooling2d_2 (MaxPooling 2	(None, 8, 8, 32)	0
conv2d_3 (Conv2D)	(None, 8, 8, 64)	8256
max_pooling2d_3 (MaxPooling 2	(None, 4, 4, 64)	0
dropout_1 (Dropout)	(None, 4, 4, 64)	0
flatten_1 (Flatten)	(None, 1024)	0
dense_1 (Dense)	(None, 500)	512500
dropout_2 (Dropout)	(None, 500)	0
dense_2 (Dense)	(None, 10)	5010

Total params: 528,054
Trainable params: 528,054
Non-trainable params: 0

9.1 딥 러닝을 이용한 이미지 분류

- 컬러 이미지 분류 문제
  - 컬러 이미지 분류하기 위한 단계
    - 4단계 : 모델 컴파일 하기
      - 모델의 학습을 시작하기 전에 손실함수, 최적화 알고리즘, 학습 과정 모니터링에 사용할 평가지표 등을 설정해야 함
      - 손실 함수 : 학습 데이터를 대상으로 신경망의 성능을 파악하는 측정 수단
      - 최적화 알고리즘 : 손실 값이 최소가 되도록 파라미터 (가중치와 편향)를 최적화하기 위해 사용할 알고리즘
      - 평가지표 : 학습 및 테스트 과정에서 사용할 모델의 평가지표, 정확도를 의미하는 metrics=['accuracy']



- 소스 코드 구현

9.1 딥 러닝을 이용한 이미지 분류

- 컬러 이미지 분류 문제
  - 컬러 이미지 분류하기 위한 단계
    - 5단계 : 모델 학습하기
      - 케라스에서는 .fit()를 호출하여 학습이 진행됨
      - 학습이 시작되고 한 에포크가 실행될때마다 메시지가 출력됨

```
from keras.callbacks import ModelCheckpoint

checkpointer = ModelCheckpoint(filepath='model.weights.best.hdf5', verbose=1,
    save_best_only=True)

hist = model.fit(x_train, y_train, batch_size=32, epochs=100,
    validation_data=(x_valid, y_valid), callbacks=[checkpointer],
    verbose=2, shuffle=True)
```

```
Train on 45000 amples, validation 5000 samples
Epoch 1/100
Epoch 00000: val_loss improved from inf to 1.35820, saving model to model.weights.best.hdf5
46s - loss: 1.6192 - acc: 0.4140 - val_loss: 1.3582 - val_acc: 0.5166
Epoch 2/100
Epoch 00001: val_loss improved from 1.35820 to 1.22245, saving model to model.weights.best.hdf5
53s - loss: 1.2881 - acc: 0.5402 - val_loss: 1.2224 - val_acc: 0.5644
Epoch 3/100
Epoch 00002: val_loss improved from 1.22245 to 1.12096, saving model to model.weights.best.hdf5
49s - loss: 1.1630 - acc: 0.5879 - val_loss: 1.1210 - val_acc: 0.6046
Epoch 4/100
Epoch 00003: val_loss improved from 1.12096 to 1.10724, saving model to model.weights.best.hdf5
56s - loss: 1.0928 - acc: 0.6160 - val_loss: 1.1072 - val_acc: 0.6134
Epoch 5/100
Epoch 00004: val_loss improved from 1.10724 to 0.97377, saving model to model.weights.best.hdf5
52s - loss: 1.0413 - acc: 0.6382 - val_loss: 0.9738 - val_acc: 0.6596
Epoch 6/100
Epoch 00005: val_loss improved from 0.97377 to 0.95501, saving model to model.weights.best.hdf5
50s - loss: 1.0090 - acc: 0.6484 - val_loss: 0.9550 - val_acc: 0.6768
Epoch 7/100
Epoch 00006: val_loss improved from 0.95501 to 0.94448, saving model to model.weights.best.hdf5
49s - loss: 0.9967 - acc: 0.6561 - val_loss: 0.9445 - val_acc: 0.6828
Epoch 8/100
Epoch 00007: val_loss did not improve
61s - loss: 0.9934 - acc: 0.6604 - val_loss: 1.1300 - val_acc: 0.6376
Epoch 9/100
Epoch 00008: val_loss improved from 0.94448 to 0.91779, saving model to model.weights.best.hdf5
49s - loss: 0.9858 - acc: 0.6672 - val_loss: 0.9178 - val_acc: 0.6882
Epoch 10/100
Epoch 00009: val_loss did not improve
50s - loss: 0.9839 - acc: 0.6658 - val_loss: 0.9669 - val_acc: 0.6748
Epoch 11/100
Epoch 00010: val_loss improved from 0.91779 to 0.91570, saving model to model.weights.best.hdf5
49s - loss: 1.0002 - acc: 0.6624 - val_loss: 0.9157 - val_acc: 0.6936
Epoch 12/100
Epoch 00011: val_loss did not improve
54s - loss: 1.0001 - acc: 0.6659 - val_loss: 1.1442 - val_acc: 0.6646
Epoch 13/100
Epoch 00012: val_loss did not improve
56s - loss: 1.0161 - acc: 0.6633 - val_loss: 0.9782 - val_acc: 0.6788
```

9.1 딥 러닝을 이용한 이미지 분류

- 5단계 : 모델 학습하기 해석
  - Loss 와 acc는 각각 학습 데이터에 대한 오차와 정확도를 의미
  - Val_loss와 val_acc는 검증 데이터에 대한 오차와 정확도
  - 한 에포크가 끝날때마다 val_loss와 val_acc의 변화를 주시, Val_loss 감소하고, val_acc가 증가해야 함
  - 가장 성능이 좋았던 가중치는 해당 에포크가 끝나면 저장되도록 구현함

9.1 딥 러닝을 이용한 이미지 분류

- 컬러 이미지 분류 문제
  - 컬러 이미지 분류하기 위한 단계
    - 6단계: val acc 가 높은 모델 사용
      - 학습이 끝났으니 load_weights() 메서드를 이용하여 val_acc 성능이 가장 좋았던 가중치를 읽어 들임
      - 케라스 소스코드

```
model.load_weights('model.weights.best.hdf5')
```

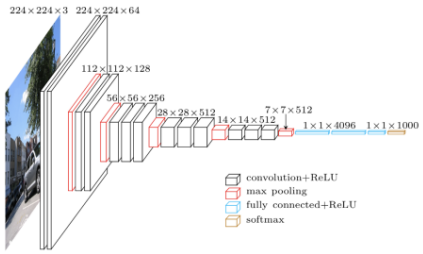
9.1 딥 러닝을 이용한 이미지 분류

- 컬러 이미지 분류 문제
  - 컬러 이미지 분류하기 위한 단계
    - 7단계 : 모델 평가하기
      - 모델을 평가하고 정확도를 계산해서 모델이 이미지 분류를 올바르게 예측하는 빈도를 나타내는 백분율을 계산함

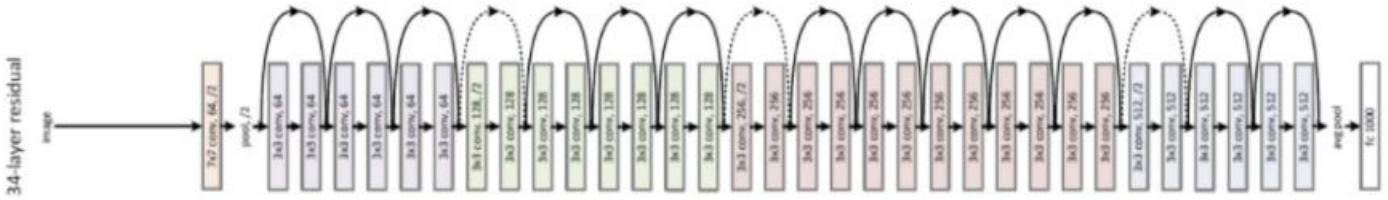
```
score = model.evaluate(x_test, y_test, verbose=0)
print('\n', 'Test accuracy:', score[1])
```

9.1 딥 러닝을 이용한 이미지 분류

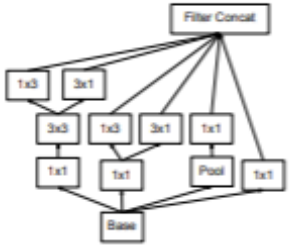
- 이미지 분류에서 유명한 딥러닝 모델 :
  - VGGNet -> RestNet -> Inception -> DenseNet -> MobileNet -> EfficientNet -> ViT



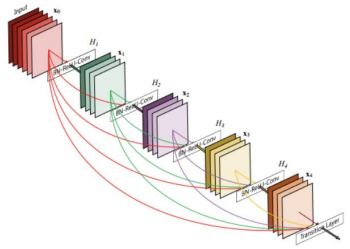
VGGNet



ResNet



Inception



DenseNet

9.2 VGGNet 이란 (1/2)

- VGGNet :
  - 2014년 ILSVRC에서 2위를 한 Oxford 대학교에서 만든 모델
  - 깊이의 영향만을 최대한 확인하고자 컨볼루션 사이즈는 가장 작은 3by3으로 고정
  - 깊이의 따른 성능 변화를 비교하기 위해 6가지 방법 모델을 제안 실험
  - 구현 : Keras Applications 에서 VGG16 으로 제공을 해줌

ConvNet Configuration					
A	A-LRN	B	C	D	E
11 weight layers	11 weight layers	13 weight layers	16 weight layers	16 weight layers	19 weight layers
input (224 × 224 RGB image)					
conv3-64	conv3-64 **LRN**	conv3-64 **conv3-64**	conv3-64 conv3-64	conv3-64 conv3-64	conv3-64 conv3-64
maxpool					
conv3-128	conv3-128	conv3-128 **conv3-128**	conv3-128 conv3-128	conv3-128 conv3-128	conv3-128 conv3-128
maxpool					
conv3-256 conv3-256	conv3-256 conv3-256	conv3-256 conv3-256	conv3-256 conv3-256 **conv1-256**	conv3-256 conv3-256 **conv3-256**	conv3-256 conv3-256 conv3-256 **conv3-256**
maxpool					
conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512 **conv1-512**	conv3-512 conv3-512 **conv3-512**	conv3-512 conv3-512 conv3-512 **conv3-512**
maxpool					
conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512 **conv1-512**	conv3-512 conv3-512 **conv3-512**	conv3-512 conv3-512 conv3-512 **conv3-512**
maxpool					
FC-4096					
FC-4096					
FC-1000					
soft-max					

```
tf.keras.applications.VGG16(
    include_top=True,
    weights="imagenet",
    input_tensor=None,
    input_shape=None,
    pooling=None,
    classes=1000,
    classifier_activation="softmax",
)
```

참조 : “Very deep convolutional networks for large scale image recognition”

- 참조 : https://keras.io/api/applications/

9.2 VGGNet 이란 (2/2)

- VGGNet 코드 (계속) :

```
model = Sequential()

model.add(Conv2D(input_shape=(224,224,3),filters=64,kernel_size=
(3,3),padding="same", activation="relu"))

model.add(Conv2D(filters=64,kernel_size=(3,3),padding="same",
activation="relu"))

model.add(MaxPool2D(pool_size=(2,2),strides=(2,2)))

model.add(Conv2D(filters=128, kernel_size=(3,3), padding="same",
activation="relu"))

model.add(Conv2D(filters=128, kernel_size=(3,3), padding="same",
activation="relu"))

model.add(MaxPool2D(pool_size=(2,2),strides=(2,2)))

model.add(Conv2D(filters=256, kernel_size=(3,3), padding="same",
activation="relu"))

model.add(Conv2D(filters=256, kernel_size=(3,3), padding="same",
activation="relu"))

model.add(Conv2D(filters=256, kernel_size=(3,3), padding="same",
activation="relu"))

model.add(MaxPool2D(pool_size=(2,2),strides=(2,2)))
```

```
model.add(Conv2D(filters=512, kernel_size=(3,3), padding="same",
activation="relu"))

model.add(Conv2D(filters=512, kernel_size=(3,3), padding="same",
activation="relu"))

model.add(Conv2D(filters=512, kernel_size=(3,3), padding="same",
activation="relu"))

model.add(MaxPool2D(pool_size=(2,2),strides=(2,2)))

model.add(Conv2D(filters=512, kernel_size=(3,3), padding="same",
activation="relu"))

model.add(Conv2D(filters=512, kernel_size=(3,3), padding="same",
activation="relu"))

model.add(Conv2D(filters=512, kernel_size=(3,3), padding="same",
activation="relu"))

model.add(MaxPool2D(pool_size=(2,2),strides=(2,2)))
```

```
model.add(Flatten())

model.add(Dense(units=4096,activation="relu"))

model.add(Dense(units=4096,activation="relu"))

model.add(Dense(units=2, activation="softmax"))
```

- 참조 : https://towardsdatascience.com/step-by-step-vgg16-implementation-in-keras-for-beginners-a833c686ae6c

9.2 ResNet 이란 (1/3)

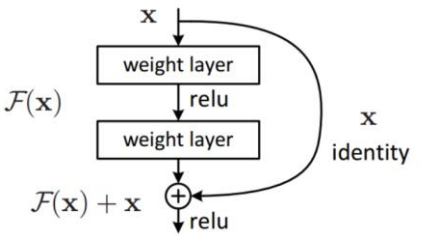
- ResNet :
  - 2015년 ILSVRC에서 1위를 한 모델
  - 핵심 내용:
    - 연산량 줄이기 위해 1by 1을 결합한 block 제안
    - 기울기 소실과 폭발을 해결하기 위해 Skip connect 을 제안

layer name	output size	18-layer	34-layer	50-layer	101-layer	152-layer
conv1	112×112	7×7, 64, stride 2				
conv2_x	56×56	3×3 max pool, stride 2				
		$\begin{bmatrix} 3\times3, 64 \\ 3\times3, 64 \end{bmatrix}\times2$	$\begin{bmatrix} 3\times3, 64 \\ 3\times3, 64 \end{bmatrix}\times3$	$\begin{bmatrix} 1\times1, 64 \\ 3\times3, 64 \\ 1\times1, 256 \end{bmatrix}\times3$	$\begin{bmatrix} 1\times1, 64 \\ 3\times3, 64 \\ 1\times1, 256 \end{bmatrix}\times3$	$\begin{bmatrix} 1\times1, 64 \\ 3\times3, 64 \\ 1\times1, 256 \end{bmatrix}\times3$
conv3_x	28×28	$\begin{bmatrix} 3\times3, 128 \\ 3\times3, 128 \end{bmatrix}\times2$	$\begin{bmatrix} 3\times3, 128 \\ 3\times3, 128 \end{bmatrix}\times4$	$\begin{bmatrix} 1\times1, 128 \\ 3\times3, 128 \\ 1\times1, 512 \end{bmatrix}\times4$	$\begin{bmatrix} 1\times1, 128 \\ 3\times3, 128 \\ 1\times1, 512 \end{bmatrix}\times4$	$\begin{bmatrix} 1\times1, 128 \\ 3\times3, 128 \\ 1\times1, 512 \end{bmatrix}\times8$
conv4_x	14×14	$\begin{bmatrix} 3\times3, 256 \\ 3\times3, 256 \end{bmatrix}\times2$	$\begin{bmatrix} 3\times3, 256 \\ 3\times3, 256 \end{bmatrix}\times6$	$\begin{bmatrix} 1\times1, 256 \\ 3\times3, 256 \\ 1\times1, 1024 \end{bmatrix}\times6$	$\begin{bmatrix} 1\times1, 256 \\ 3\times3, 256 \\ 1\times1, 1024 \end{bmatrix}\times23$	$\begin{bmatrix} 1\times1, 256 \\ 3\times3, 256 \\ 1\times1, 1024 \end{bmatrix}\times36$
conv5_x	7×7	$\begin{bmatrix} 3\times3, 512 \\ 3\times3, 512 \end{bmatrix}\times2$	$\begin{bmatrix} 3\times3, 512 \\ 3\times3, 512 \end{bmatrix}\times3$	$\begin{bmatrix} 1\times1, 512 \\ 3\times3, 512 \\ 1\times1, 2048 \end{bmatrix}\times3$	$\begin{bmatrix} 1\times1, 512 \\ 3\times3, 512 \\ 1\times1, 2048 \end{bmatrix}\times3$	$\begin{bmatrix} 1\times1, 512 \\ 3\times3, 512 \\ 1\times1, 2048 \end{bmatrix}\times3$
	1×1	average pool, 1000-d fc, softmax				
FLOPs		1.8×10^9	3.6×10^9	3.8×10^9	7.6×10^9	11.3×10^9

제안한 구조

```
tf.keras.applications.ResNet50(
    include_top=True,
    weights="imagenet",
    input_tensor=None,
    input_shape=None,
    pooling=None,
    classes=1000,
    **kwargs
)
```

ResNet API



Skip connect

참조 : https://keras.io/api/applications/resnet/#resnet50-function

•참조 : Deep Residual Learning for Image Recognition (CVPR 2015)

- **기울 기 소실**: Layer가 깊어질수록 경사가 사라지는 현상

9.2 ResNet 이란 (2/3)

- ResNet 코드 :

```
x = layers.Conv2D(filters1, (1, 1), strides=strides,
                  kernel_initializer='he_normal',
                  name=conv_name_base + '2a')(input_tensor)
x = layers.BatchNormalization(axis=bn_axis, name=bn_name_base + '2a')(x)
x = layers.Activation('relu')(x)

x = layers.Conv2D(filters2, kernel_size, padding='same',
                  kernel_initializer='he_normal',
                  name=conv_name_base + '2b')(x)
x = layers.BatchNormalization(axis=bn_axis, name=bn_name_base + '2b')(x)
x = layers.Activation('relu')(x)

x = layers.Conv2D(filters3, (1, 1),
                  kernel_initializer='he_normal',
                  name=conv_name_base + '2c')(x)
x = layers.BatchNormalization(axis=bn_axis, name=bn_name_base + '2c')(x)

shortcut = layers.Conv2D(filters3, (1, 1), strides=strides,
                         kernel_initializer='he_normal',
                         name=conv_name_base + '1')(input_tensor)
shortcut = layers.BatchNormalization(
    axis=bn_axis, name=bn_name_base + '1')(shortcut)

x = layers.add([x, shortcut])
x = layers.Activation('relu')(x)
```

Conv block

```
x = layers.Conv2D(filters1, (1, 1),
                  kernel_initializer='he_normal',
                  name=conv_name_base + '2a')(input_tensor)
x = layers.BatchNormalization(axis=bn_axis, name=bn_name_base + '2a')(x)
x = layers.Activation('relu')(x)

x = layers.Conv2D(filters2, kernel_size,
                  padding='same',
                  kernel_initializer='he_normal',
                  name=conv_name_base + '2b')(x)
x = layers.BatchNormalization(axis=bn_axis, name=bn_name_base + '2b')(x)
x = layers.Activation('relu')(x)

x = layers.Conv2D(filters3, (1, 1),
                  kernel_initializer='he_normal',
                  name=conv_name_base + '2c')(x)
x = layers.BatchNormalization(axis=bn_axis, name=bn_name_base + '2c')(x)

x = layers.add([x, input_tensor])
x = layers.Activation('relu')(x)
```

Identity block

참조 : https://github.com/keras-team/keras-applications/blob/master/keras_applications/resnet50.py

9.2 ResNet 이란 (3/3)

- ResNet 코드 (계속):

```
x = layers.ZeroPadding2D(padding=(3, 3), name='conv1_pad')(img_input)
x = layers.Conv2D(64, (7, 7),
                  strides=(2, 2),
                  padding='valid',
                  kernel_initializer='he_normal',
                  name='conv1')(x)
x = layers.BatchNormalization(axis=bn_axis, name='bn_conv1')(x)
x = layers.Activation('relu')(x)
x = layers.ZeroPadding2D(padding=(1, 1), name='pool1_pad')(x)
x = layers.MaxPooling2D((3, 3), strides=(2, 2))(x)

x = conv_block(x, 3, [64, 64, 256], stage=2, block='a', strides=(1, 1))
x = identity_block(x, 3, [64, 64, 256], stage=2, block='b')
x = identity_block(x, 3, [64, 64, 256], stage=2, block='c')

x = conv_block(x, 3, [128, 128, 512], stage=3, block='a')
x = identity_block(x, 3, [128, 128, 512], stage=3, block='b')
x = identity_block(x, 3, [128, 128, 512], stage=3, block='c')
x = identity_block(x, 3, [128, 128, 512], stage=3, block='d')

x = conv_block(x, 3, [256, 256, 1024], stage=4, block='a')
x = identity_block(x, 3, [256, 256, 1024], stage=4, block='b')
x = identity_block(x, 3, [256, 256, 1024], stage=4, block='c')
x = identity_block(x, 3, [256, 256, 1024], stage=4, block='d')
x = identity_block(x, 3, [256, 256, 1024], stage=4, block='e')
x = identity_block(x, 3, [256, 256, 1024], stage=4, block='f')
```

ResNet

```
x = conv_block(x, 3, [512, 512, 2048], stage=5, block='a')
x = identity_block(x, 3, [512, 512, 2048], stage=5, block='b')
x = identity_block(x, 3, [512, 512, 2048], stage=5, block='c')

if include_top:
    x = layers.GlobalAveragePooling2D(name='avg_pool')(x)
    x = layers.Dense(classes, activation='softmax', name='fc1000')(x)
# Create model.
model = models.Model(inputs, x, name='resnet50')
```

https://github.com/keras-team/keras-applications/blob/master/keras_applications/resnet50.py

9.2 Inception 이란 (1/2)

- Inception :
  - 컨볼루션 layer에서 사용하는 커널을 개선하여 연산의 복잡도를 줄이고 파라미터 수를 적게 사용함으로써 파라미터 및 속도개선
  - 즉, Inception module을 통해 채널 수를 조절 가능, 파라미터 수 감소

type	patch size/stride or remarks	input size
conv	3×3/2	299×299×3
conv	3×3/1	149×149×32
conv padded	3×3/1	147×147×32
pool	3×3/2	147×147×64
conv	3×3/1	73×73×64
conv	3×3/2	71×71×80
conv	3×3/1	35×35×192
3× Inception	As in figure 5	35×35×288
5× Inception	As in figure 6	17×17×768
2× Inception	As in figure 7	8×8×1280
pool	8 × 8	8 × 8 × 2048
linear	logits	1 × 1 × 2048
softmax	classifier	1 × 1 × 1000

Inception module

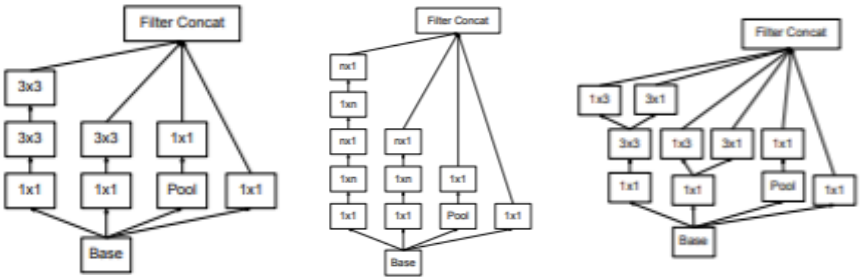


Figure 5,6,7

```
tf.keras.applications.InceptionV3(
    include_top=True,
    weights="imagenet",
    input_tensor=None,
    input_shape=None,
    pooling=None,
    classes=1000,
    classifier_activation="softmax",
)
```

https://keras.io/api/applications/inceptionv3/

•Rethinking the Inception Architecture for Computer Vision (CVPR 2016)

9.2 Inception 이란 (2/2)

- Inception 코드 (계속):

```
# Determine proper input shape
input_shape = _obtain_input_shape(
    input_shape,
    default_size=299,
    min_size=75,
    data_format=backend.image_data_format(),
    require_flatten=include_top,
    weights=weights)

if input_tensor is None:
    img_input = layers.Input(shape=input_shape)
else:
    if not backend.is_keras_tensor(input_tensor):
        img_input = layers.Input(tensor=input_tensor, shape=input_shape)
    else:
        img_input = input_tensor

if backend.image_data_format() == 'channels_first':
    channel_axis = 1
else:
    channel_axis = 3

x = conv2d_bn(img_input, 32, 3, 3, strides=(2, 2), padding='valid')
x = conv2d_bn(x, 32, 3, 3, padding='valid')
x = conv2d_bn(x, 64, 3, 3)
x = layers.MaxPooling2D((3, 3), strides=(2, 2))(x)

x = conv2d_bn(x, 80, 1, 1, padding='valid')
x = conv2d_bn(x, 192, 3, 3, padding='valid')
x = layers.MaxPooling2D((3, 3), strides=(2, 2))(x)
```

```
# mixed 0: 35 x 35 x 256
branch1x1 = conv2d_bn(x, 64, 1, 1)

branch5x5 = conv2d_bn(x, 48, 1, 1)
branch5x5 = conv2d_bn(branch5x5, 64, 5, 5)

branch3x3dbl = conv2d_bn(x, 64, 1, 1)
branch3x3dbl = conv2d_bn(branch3x3dbl, 96, 3, 3)
branch3x3dbl = conv2d_bn(branch3x3dbl, 96, 3, 3)

branch_pool = layers.AveragePooling2D((3, 3),
                                      strides=(1, 1),
                                      padding='same')(x)
branch_pool = conv2d_bn(branch_pool, 32, 1, 1)
x = layers.concatenate(
    [branch1x1, branch5x5, branch3x3dbl, branch_pool],
    axis=channel_axis,
    name='mixed0')
```

```
# mixed 1: 35 x 35 x 288
branch1x1 = conv2d_bn(x, 64, 1, 1)

branch5x5 = conv2d_bn(x, 48, 1, 1)
branch5x5 = conv2d_bn(branch5x5, 64, 5, 5)

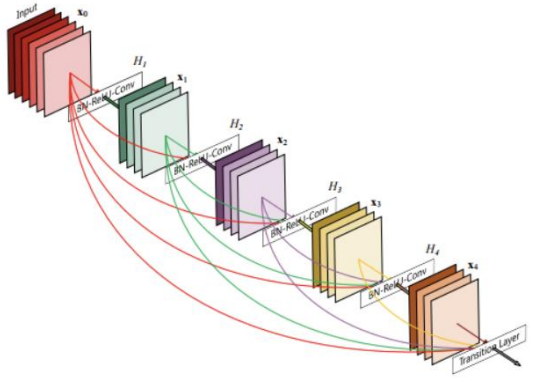
branch3x3dbl = conv2d_bn(x, 64, 1, 1)
branch3x3dbl = conv2d_bn(branch3x3dbl, 96, 3, 3)
branch3x3dbl = conv2d_bn(branch3x3dbl, 96, 3, 3)

branch_pool = layers.AveragePooling2D((3, 3),
                                      strides=(1, 1),
                                      padding='same')(x)
```

https://github.com/keras-team/keras-applications/blob/master/keras_applications/inception_v3.py

9.2 DenseNet 이란 (1/2)

- DenseNet :
  - 모든 레이어의 특징맵을 연결하는 방법 (Dense block)
  - 특징맵을 계속해서 연결하면 채널 수가 많아 질 수 있기 때문에 특징맵 수는 적게 사용



Dense block

Layers	Output Size	DenseNet-121	DenseNet-169	DenseNet-201	DenseNet-264
Convolution	112 × 112	7 × 7 conv, stride 2			
Pooling	56 × 56	3 × 3 max pool, stride 2			
Dense Block (1)	56 × 56	[1 × 1 conv, 3 × 3 conv] × 6	[1 × 1 conv, 3 × 3 conv] × 6	[1 × 1 conv, 3 × 3 conv] × 6	[1 × 1 conv, 3 × 3 conv] × 6
Transition Layer (1)	56 × 56	1 × 1 conv			
	28 × 28	2 × 2 average pool, stride 2			
Dense Block (2)	28 × 28	[1 × 1 conv, 3 × 3 conv] × 12	[1 × 1 conv, 3 × 3 conv] × 12	[1 × 1 conv, 3 × 3 conv] × 12	[1 × 1 conv, 3 × 3 conv] × 12
Transition Layer (2)	28 × 28	1 × 1 conv			
	14 × 14	2 × 2 average pool, stride 2			
Dense Block (3)	14 × 14	[1 × 1 conv, 3 × 3 conv] × 24	[1 × 1 conv, 3 × 3 conv] × 32	[1 × 1 conv, 3 × 3 conv] × 48	[1 × 1 conv, 3 × 3 conv] × 64
Transition Layer (3)	14 × 14	1 × 1 conv			
	7 × 7	2 × 2 average pool, stride 2			
Dense Block (4)	7 × 7	[1 × 1 conv, 3 × 3 conv] × 16	[1 × 1 conv, 3 × 3 conv] × 32	[1 × 1 conv, 3 × 3 conv] × 32	[1 × 1 conv, 3 × 3 conv] × 48
Classification Layer	1 × 1	7 × 7 global average pool			
		1000D fully-connected, softmax			

DenseNet 구조

```
tf.keras.applications.DenseNet121(
    include_top=True,
    weights="imagenet",
    input_tensor=None,
    input_shape=None,
    pooling=None,
    classes=1000,
)
```

https://keras.io/api/applications/densenet/#densenet121-function

9.2 DenseNet 이란 (2/2)

- DenseNet 코드 (계속) :

```
x = layers.ZeroPadding2D(padding=((3, 3), (3, 3)))(img_input)
x = layers.Conv2D(64, 7, strides=2, use_bias=False, name='conv1/conv')(
x = layers.BatchNormalization(
    axis=bn_axis, epsilon=1.001e-5, name='conv1/bn')(x)
x = layers.Activation('relu', name='conv1/relu')(x)
x = layers.ZeroPadding2D(padding=((1, 1), (1, 1)))(x)
x = layers.MaxPooling2D(3, strides=2, name='pool1')(x)

x = dense_block(x, blocks[0], name='conv2')
x = transition_block(x, 0.5, name='pool2')
x = dense_block(x, blocks[1], name='conv3')
x = transition_block(x, 0.5, name='pool3')
x = dense_block(x, blocks[2], name='conv4')
x = transition_block(x, 0.5, name='pool4')
x = dense_block(x, blocks[3], name='conv5')
```

Dense block

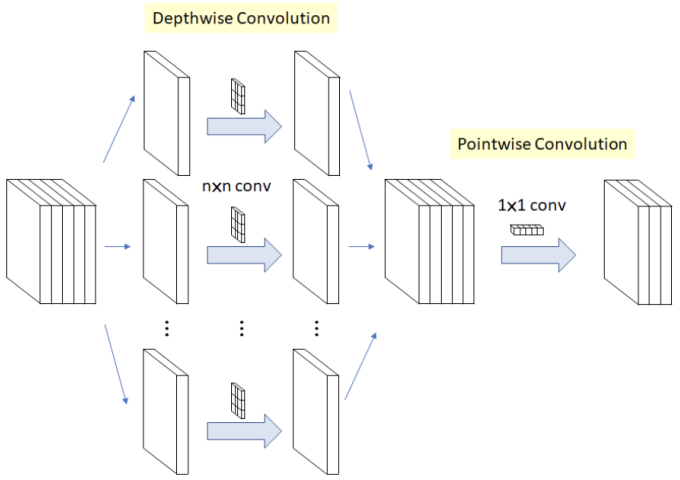
```
x = layers.BatchNormalization(
    axis=bn_axis, epsilon=1.001e-5, name='bn')(x)
x = layers.Activation('relu', name='relu')(x)

if include_top:
    x = layers.GlobalAveragePooling2D(name='avg_pool')(x)
    x = layers.Dense(classes, activation='softmax', name='fc1000')(x)
else:
    if pooling == 'avg':
        x = layers.GlobalAveragePooling2D(name='avg_pool')(x)
    elif pooling == 'max':
        x = layers.GlobalMaxPooling2D(name='max_pool')(x)
```

https://github.com/keras-team/keras-applications/blob/master/keras_applications/densenet.py

9.2 MobileNet 이란 (1/2)

- MobileNet :
  - 목표 :
    - 모바일 기기에서 서빙 가능한 가벼운 모델 연구
  - 핵심 :
    - Depthwise Convolution : 공간 방향으로 Feature map 의 각 채널마다 각기 다른 커널을 사용
    - Pointwise Convolution (1by 1Con volution) : 채널 방향으로 1by 1 컨볼루션
    - Depthwise Separable Convolution : Depthwise Convolution + Pointwise Convolution



depth wise separable convolution

•MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications

https://keras.io/api/applications/mobilenet/

9.2 MobileNet 이란 (2/2)

- MobileNet 코드 (계속):

```
x = _conv_block(img_input, 32, alpha, strides=(2, 2))
x = _depthwise_conv_block(x, 64, alpha, depth_multiplier, block_id=1)

x = _depthwise_conv_block(x, 128, alpha, depth_multiplier,
                          strides=(2, 2), block_id=2)
x = _depthwise_conv_block(x, 128, alpha, depth_multiplier, block_id=3)

x = _depthwise_conv_block(x, 256, alpha, depth_multiplier,
                          strides=(2, 2), block_id=4)
x = _depthwise_conv_block(x, 256, alpha, depth_multiplier, block_id=5)

x = _depthwise_conv_block(x, 512, alpha, depth_multiplier,
                          strides=(2, 2), block_id=6)
x = _depthwise_conv_block(x, 512, alpha, depth_multiplier, block_id=7)
x = _depthwise_conv_block(x, 512, alpha, depth_multiplier, block_id=8)
x = _depthwise_conv_block(x, 512, alpha, depth_multiplier, block_id=9)
x = _depthwise_conv_block(x, 512, alpha, depth_multiplier, block_id=10)
x = _depthwise_conv_block(x, 512, alpha, depth_multiplier, block_id=11)

x = _depthwise_conv_block(x, 1024, alpha, depth_multiplier,
                          strides=(2, 2), block_id=12)
x = _depthwise_conv_block(x, 1024, alpha, depth_multiplier, block_id=13)

if include_top:
    if backend.image_data_format() == 'channels_first':
        shape = (int(1024 * alpha), 1, 1)
    else:
        shape = (1, 1, int(1024 * alpha))

    x = layers.GlobalAveragePooling2D()(x)
```

DenseNet

```
if strides == (1, 1):
    x = inputs
else:
    x = layers.ZeroPadding2D(((0, 1), (0, 1)),
                             name='conv_pad_%d' % block_id)(inputs)
x = layers.DepthwiseConv2D((3, 3),
                           padding='same' if strides == (1, 1) else 'valid',
                           depth_multiplier=depth_multiplier,
                           strides=strides,
                           use_bias=False,
                           name='conv_dw_%d' % block_id)(x)
x = layers.BatchNormalization(
    axis=channel_axis, name='conv_dw_%d_bn' % block_id)(x)
x = layers.ReLU(6., name='conv_dw_%d_relu' % block_id)(x)

x = layers.Conv2D(pointwise_conv_filters, (1, 1),
                  padding='same',
                  use_bias=False,
                  strides=(1, 1),
                  name='conv_pw_%d' % block_id)(x)
x = layers.BatchNormalization(axis=channel_axis,
                              name='conv_pw_%d_bn' % block_id)(x)
return layers.ReLU(6., name='conv_pw_%d_relu' % block_id)(x)
```

Depthwise Separable Convolution

https://github.com/keras-team/keras-applications/blob/master/keras_applications/mobilenet.py

9.2 EfficientNet 이란 (1/2)

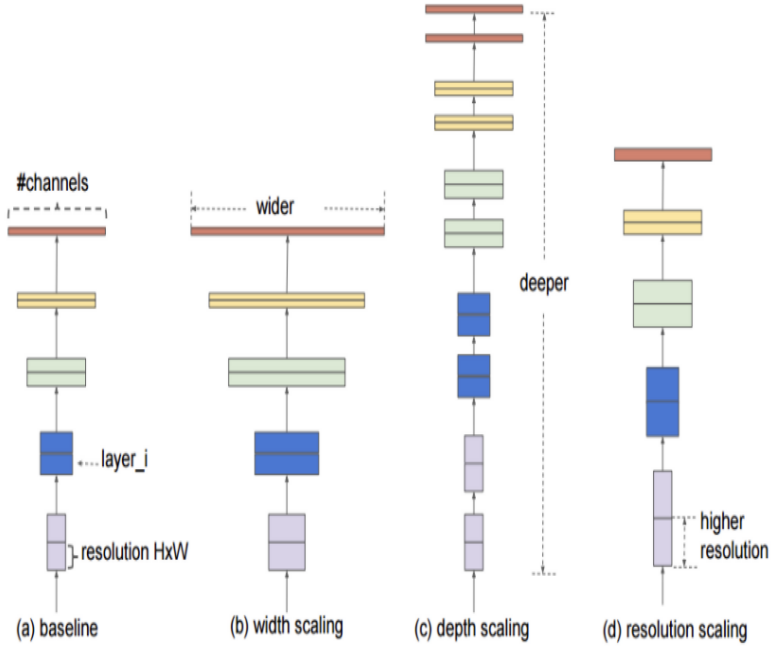
- EfficientNet :
  - 모델의 깊이, 너비, 입력 이미지의 크기를 조절
  - 3가지를 효율적으로 조절할 수 있는 Compound Scaling 방법을 제안
  - Compound Scaling :

$$\text{depth: } d = \alpha^{\phi}$$

$$\text{width: } w = \beta^{\phi}$$

$$\text{resolution: } r = \gamma^{\phi}$$

Small grid search 로 결정되는 상수(알파, 베타, 감마)와 주어진 연산량에 따라 사용자가 결정하는 상수(파이)로 구성




```
tf.keras.applications.EfficientNetB0(
    include_top=True,
    weights="imagenet",
    input_tensor=None,
    input_shape=None,
    pooling=None,
    classes=1000,
    classifier_activation="softmax",
    **kwargs
)
```

https://keras.io/api/applications/efficientnet/#efficientnetb0-function

9.2 EfficientNet 이란 (2/2)

- EfficinetNet 코드 (계속):

```python
def round_filters(filters, divisor=depth_divisor):
    """Round number of filters based on depth multiplier."""
    filters *= width_coefficient
    new_filters = max(divisor, int(filters + divisor / 2) // divisor * divisor)
    # Make sure that round down does not go down by more than 10%.
    if new_filters < 0.9 * filters:
        new_filters += divisor
    return int(new_filters)

def round_repeats(repeats):
    """Round number of repeats based on depth multiplier."""
    return int(math.ceil(depth_coefficient * repeats))
```

넓이 관련 요소

```python
DEFAULT_BLOCKS_ARGS = [
    {'kernel_size': 3, 'repeats': 1, 'filters_in': 32, 'filters_out': 16,
     'expand_ratio': 1, 'id_skip': True, 'strides': 1, 'se_ratio': 0.25},
    {'kernel_size': 3, 'repeats': 2, 'filters_in': 16, 'filters_out': 24,
     'expand_ratio': 6, 'id_skip': True, 'strides': 2, 'se_ratio': 0.25},
    {'kernel_size': 5, 'repeats': 2, 'filters_in': 24, 'filters_out': 40,
     'expand_ratio': 6, 'id_skip': True, 'strides': 2, 'se_ratio': 0.25},
    {'kernel_size': 3, 'repeats': 3, 'filters_in': 40, 'filters_out': 80,
     'expand_ratio': 6, 'id_skip': True, 'strides': 2, 'se_ratio': 0.25},
    {'kernel_size': 5, 'repeats': 3, 'filters_in': 80, 'filters_out': 112,
     'expand_ratio': 6, 'id_skip': True, 'strides': 1, 'se_ratio': 0.25},
    {'kernel_size': 5, 'repeats': 4, 'filters_in': 112, 'filters_out': 192
     'expand_ratio': 6, 'id_skip': True, 'strides': 2, 'se_ratio': 0.25},
    {'kernel_size': 3, 'repeats': 1, 'filters_in': 192, 'filters_out': 320
     'expand_ratio': 6, 'id_skip': True, 'strides': 1, 'se_ratio': 0.25}
]
```

깊이 관련 요소

```python
# Build top
x = layers.Conv2D(round_filters(1280), 1,
                  padding='same',
                  use_bias=False,
                  kernel_initializer=CONV_KERNEL_INITIALIZER,
                  name='top_conv')(x)
x = layers.BatchNormalization(axis=bn_axis, name='top_bn')(x)
x = layers.Activation(activation_fn, name='top_activation')(x)
if include_top:
    x = layers.GlobalAveragePooling2D(name='avg_pool')(x)
    if dropout_rate > 0:
        x = layers.Dropout(dropout_rate, name='top_dropout')(x)
    x = layers.Dense(classes,
                     activation='softmax',
                     kernel_initializer=DENSE_KERNEL_INITIALIZER,
                     name='probs')(x)
else:
    if pooling == 'avg':
        x = layers.GlobalAveragePooling2D(name='avg_pool')(x)
    elif pooling == 'max':
        x = layers.GlobalMaxPooling2D(name='max_pool')(x)

# Ensure that the model takes into account
# any potential predecessors of `input_tensor`.
if input_tensor is not None:
    inputs = keras_utils.get_source_inputs(input_tensor)
else:
    inputs = img_input

# Create model.
model = models.Model(inputs, x, name=model_name)
```

EfficientNet

https://github.com/keras-team/keras-applications/blob/master/keras_applications/efficientnet.py

9.3 이미지 분류 모델 구현

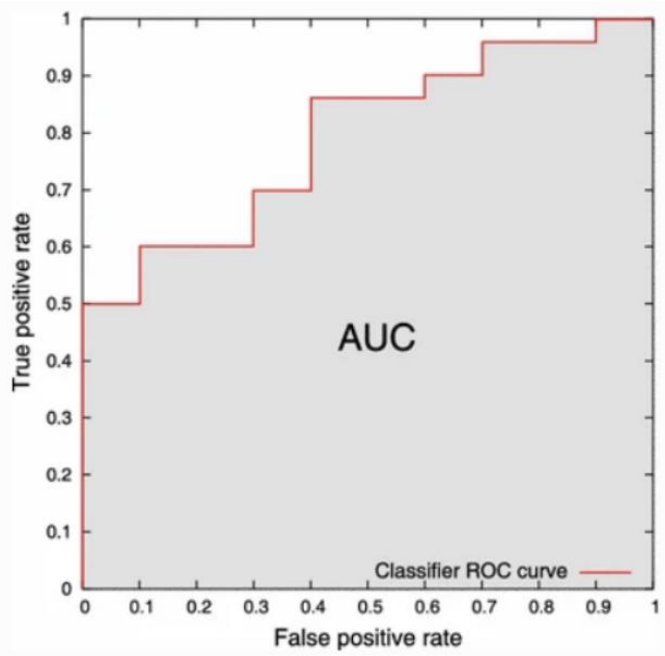
- 이미지 분류 딥 러닝 모델 구현 과정 포인트 :
  - 컨볼루션 등을 이용해 추출된 특징 맵에서 분류 할 수 있도록 특징 맵 모양 변환 과정
  - 클래스 수에 따른 딥 러닝 모델 Output 사이즈 설정

9.4 정확도 측정 방법 (1/2)

- 평가 방법 :
  - Confusion matrix : 이전 수업 자료에 기재
  - Top 1 Accuracy : 가장 높은 1개의 정확도
  - Top 5 Accuracy : 가장 높은 5개의 정확도
  - Sensitivity :
    - 질병(False)있는 사람을 얼마나 잘 찾아 내는가에 대한 값
  - Specificity :
    - 정상(True)을 얼마나 잘 찾아내는가에 대한 값

9.4 정확도 측정 방법 (2/2)

- 평가 방법 :
  - AUC :
    - ROC 곡선 아래 영역
    - 높다는 의미는 클래스를 구별하는 모델의 성능이 좋은 것
  - ROC Curve :
    - 모든 임계 값에서 분류 모델의 성능을 보여주는 그래프
    - ROC 커브에서 왼쪽 상단으로 가면 갈 수록 좋은 것



9.5 이미지 분류 오픈 데이터 (1/2)

- 데이터 세트 :
  - Cifar 10 (*)
  - Cifar 100
  - Image Net
  - MNIST (*)
  - Fashion MNIST
  - SVHN (*)
  - STL-10

SVHN (Street View House Numbers)

Edit

Introduced by Netzer et al. in Reading digits in natural images with unsupervised feature learning

Street View House Numbers (**SVHN**) is a digit classification benchmark dataset that contains 600,000 32×32 RGB images of printed digits (from 0 to 9) cropped from pictures of house number plates. The cropped images are centered in the digit of interest, but nearby digits and other distractors are kept in the image. SVHN has three sets: training, testing sets and an extra set with 530,000 images that are less difficult and can be used for helping with the training process.

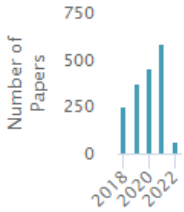


Source: http://ufldl.stanf...

Source: Competitive Multi-scale Convolution

Homepage

Usage



9.5 이미지 분류 오픈 데이터 (2/2)

- 데이터 세트 :
  - Cifar 10 (*)
  - Cifar 100
  - Image Net
  - MNIST (*)
  - Fashion MNIST
  - SVHN (*)
  - STL-10

CIFAR-10

Edit

Introduced by Krizhevsky et al. in Learning multiple layers of features from tiny images

The **CIFAR-10** dataset (Canadian Institute for Advanced Research, 10 classes) is a subset of the Tiny Images dataset and consists of 60000 32x32 color images. The images are labelled with one of 10 mutually exclusive classes: airplane, automobile (but not truck or pickup truck), bird, cat, deer, dog, frog, horse, ship, and truck (but not pickup truck). There are 6000 images per class with 5000 training and 1000 testing images per class.



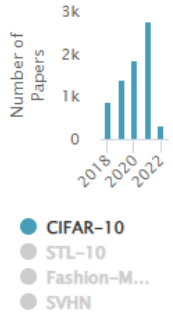
Source: https://www.cs.t...

The criteria for deciding whether an image belongs to a class were as follows:

- The class name should be high on the list of likely answers to the question "What is in this picture?"
- The image should be photo-realistic. Labelers were instructed to reject line drawings.
- The image should contain only one prominent instance of the object to which the class refers. The object may be partially occluded or seen from an unusual viewpoint as long as its identity is still clear to the labeler.

Source: https://www.cs.toronto.edu/~kriz/cifar.html

Usage



9.6 현재 이미지 분류 최신 모델

- 현 시점 Image Net 기준 가장 성능 좋은 모델 :
  - 1. CoAtNet-7
  - 2. ViT-G/14
  - 3. CoAtNet-6
  - 4. V-MoE-15B
  - 5. Meta Pseudo Labels
  - 6. SwinV2-G
    - 참조 : https://paperswithcode.com/sota/image-classification-on-imagenet

질문 및 정리

